

# React Apps





# React

- **What is React?**
  - A JavaScript library for building user interfaces
  - Originally developed by Facebook in 2011, open-sourced in 2013
  - Designed for building fast, interactive web apps
- **Why do people like it?**
  - Looks really cool
  - Component-based architecture → Reusable, modular code
  - Virtual DOM → Fast updates & rendering
- **Where is React used?**
  - Facebook, Instagram, Netflix, Airbnb, Uber, and thousands of startups

# Node.js

- Node.js is a runtime environment that lets you run **JavaScript** on your computer, not just in the browser
- Lets developers use JavaScript for backend servers, command-line tools, and more
- React is built with modern JavaScript tools that require Node.js to work
- Download here: <https://nodejs.org/en/download>
  - Scroll down and choose “prebuilt Node.js”

# Vite

- **Vite is a modern frontend build tool and development server**
- **We use Vite to set up and run the React app**
- **Vite = fast in French**
- **React apps run in Vite start quickly**

# Vite React App Structure

```
my-vite-app/
├── public/                ← Static assets (icons, robots.txt)
│   └── vite.svg
├── src/
│   ├── assets/           ← Images/styles to be processed by Vite
│   ├── components/       ← Your custom React components
│   │   └── MyComponent.tsx
│   ├── App.css           ← Styles for App component
│   ├── App.tsx           ← Root React component
│   ├── main.tsx          ← React entry point (replaces index.js)
│   └── vite-env.d.ts     ← TypeScript definitions for Vite
├── index.html            ← The entry point for the browser (now in root!)
├── package.json          ← Project metadata and dependencies
├── tsconfig.json         ← TypeScript configuration
└── vite.config.ts        ← Vite-specific settings/plugins
```

# Creating a Vite React App

1. Make a folder (i.e. my-app)
2. Open the folder in Cursor (so your terminal is in the folder)
3. Scaffold the app - In the terminal run  
**npm create vite@latest . -- --template react-ts**
  - The dot means make the app inside this folder you are in
  - --template react-ts means the language is TypeScript
4. Install – in the terminal run **npm install**
  - Downloads all React and Vite packages
5. Run the app - In the terminal, run **npm run dev**

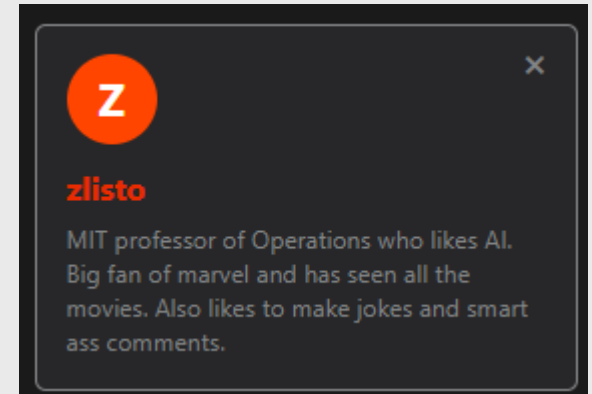
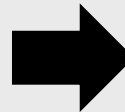
# React Components

- A component is a reusable piece of user interface
- Components are JavaScript or TypeScript files (.js, .tsx)
- React apps are built by combining components like Lego blocks
- Example: this component will produce some nice HTML formatted text that says **This is my custom component!**

```
function MyComponent() {  
  return <p>This is my custom component!</p>;  
}  
  
export default MyComponent;
```

# Example: AgentCard

```
const AgentCard = ({ agent, onDelete }) => {  
  return (  
    <div className="agent-card">  
      <div className="agent-avatar" style={{ backgroundColor: agent.color }}>  
        {agent.name.charAt(0).toUpperCase()}  
      </div>  
      <div className="agent-info">  
        <h3 style={{ color: agent.color }}>{agent.name}</h3>  
        <p>{agent.bio}</p>  
      </div>  
      <button  
        className="delete-agent"  
        onClick={() => onDelete(agent.id)}  
      >  
        ✖  
      </button>  
    </div>  
  );  
};
```



# Example: App

- App is the main component that renders everything inside your app (this is in the App.tsx file)
- This component uses other components

```
export default function App() {
  const fileInputRef = useRef<HTMLInputElement>(null)
  const [lectureDetails, setLectureDetails] = useState('')
  const [speakingStyle, setSpeakingStyle] = useState('')
  const [slides, setSlides] = useState<Slide[]>([])
  const [pdfName, setPdfName] = useState<string | null>(null)

  const [busy, setBusy] = useState<Busy>('idle')
  const [progress, setProgress] = useState(0)
  const [progressNote, setProgressNote] = useState('')
  const [error, setError] = useState<string | null>(null)

  const [dragOver, setDragOver] = useState(false)

  const geminiConfigured = Boolean(getGeminiApiKey())
  const elevenConfigured = Boolean(getElevenLabsApiKey())

  const [ttsProvider, setTtsProvider] = useState<'gemini' | 'elevenlabs'>('gemini')
  const [geminiVoice, setGeminiVoice] = useState('Kore')
  const [elevenVoices, setElevenVoices] = useState<ElevenVoice[]>([])
  const [elevenVoiceId, setElevenVoiceId] = useState(() => getDefaultElevenLabsVoiceId())

  useEffect(() => {
    if (!elevenConfigured || ttsProvider !== 'elevenlabs') return
    let cancelled = false
    void (async () => {
      try {
        const v = await fetchElevenLabsVoices(getElevenLabsApiKey())
        if (cancelled) return
        setElevenVoices(v)
        if (!getDefaultElevenLabsVoiceId() && v[0]) setElevenVoiceId(v[0].voice_id)
      } catch {
        // ignore
      }
    })()
  }, [elevenConfigured, ttsProvider])
}
```

# main.tsx

- In the main.tsx file, your App component is rendered to run your app

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

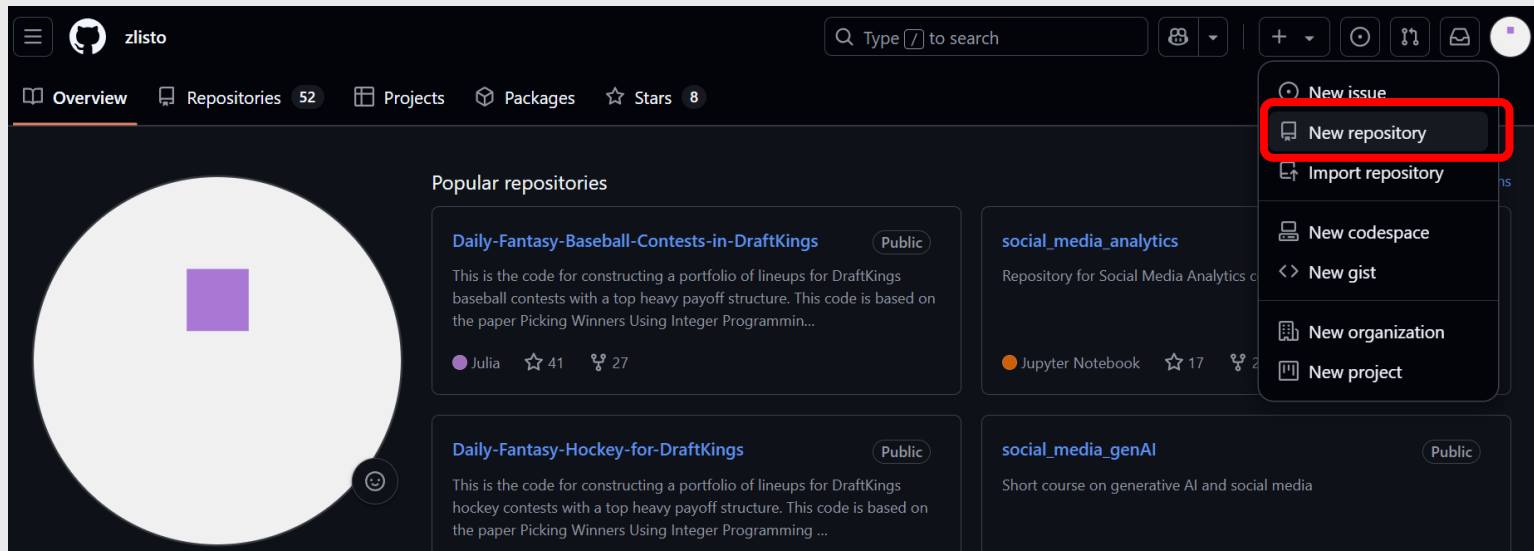
**GITHUB**

# **Setting Up Git and GitHub**

- **Once your code is done, run the app locally on your machine to make sure it works**
- **If everything is working, you are ready to push it to GitHub**
  - 1. Install Git**
  - 2. Create a GitHub account**
  - 3. Connect Cursor to your GitHub account**

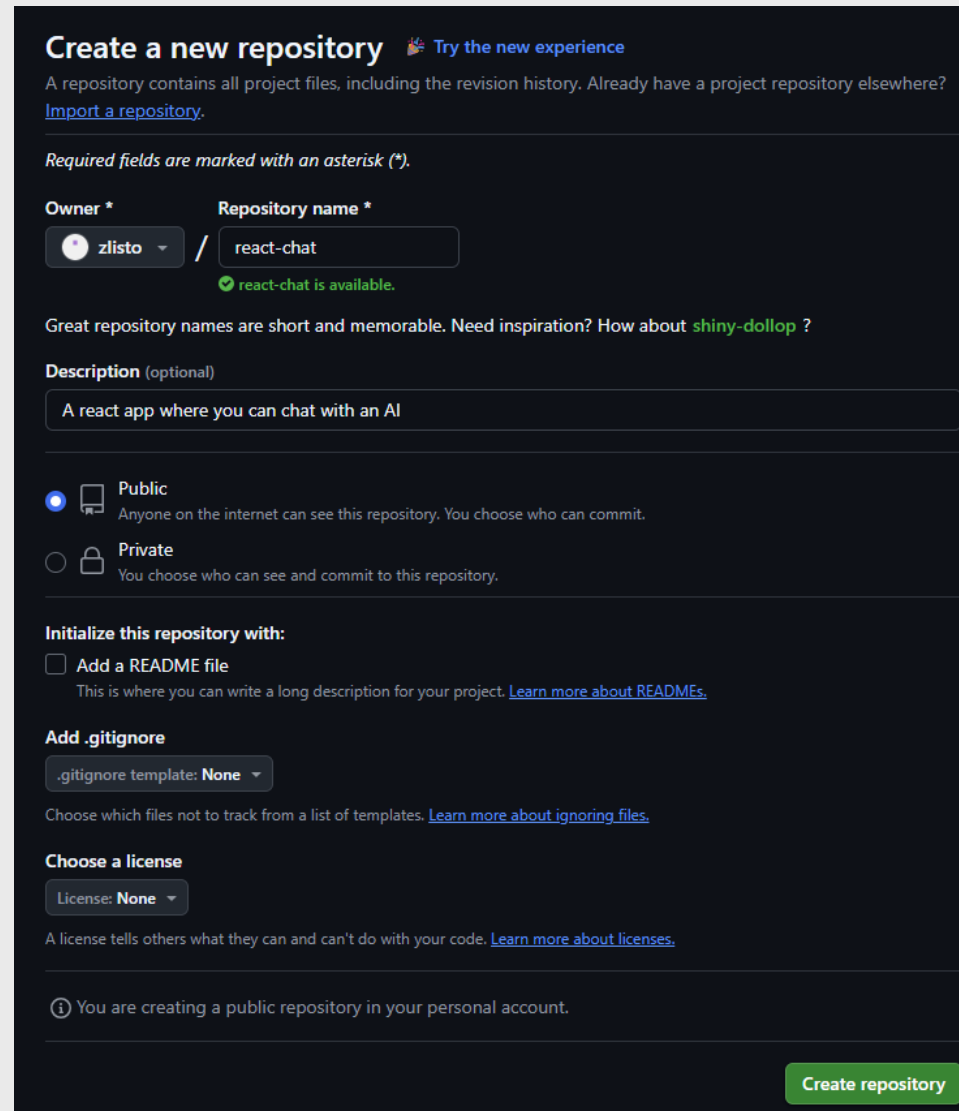
# Create GitHub Repo

- Create a “New repository” (repo) on GitHub



# Create GitHub Repo

- Choose a name for the repo
- Choose public or private
  - Private if the code will make you rich one day
- Don't add a README file
- Don't add a .gitignore file
- Don't choose a license
- We want an empty repo so we can push all our code to it
- AI will make these files for us



**Create a new repository** 🚀 [Try the new experience](#)

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

*Required fields are marked with an asterisk (\*).*

**Owner \***  zlisto / **Repository name \***

✅ react-chat is available.

Great repository names are short and memorable. Need inspiration? How about [shiny-dollop](#) ?

**Description** (optional)

☒  **Public**  
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**  
You choose who can see and commit to this repository.

**Initialize this repository with:**

☐ Add a README file  
This is where you can write a long description for your project. [Learn more about READMEs.](#)

**Add .gitignore**

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

**Choose a license**

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

📘 You are creating a public repository in your personal account.

[Create repository](#)

# Initial Commit to Empty GitHub Repo



## TERMINAL WAY

```
git init
git add .
git commit -m "initial vibe"
# Connect to the cloud:
git remote add origin [git URL]
git push -u origin main
```



## CURSOR WAY

1. **Initialize:** Source Control Tab → "Initialize Repository"
2. **Stage & Commit:** Click + then type your message and Commit.
3. **Add Remote:**
  - Click ... (Three Dots) in Git sidebar.
  - Select Remote → Add Remote.
  - Name: **origin**.
  - URL:  
<https://github.com/<user>/<repo>.git>.
4. **Push:** Click Publish Branch.



**Pro-Tip:** The URL always follows the format:

[https://github.com/YOUR\\_USERNAME/YOUR\\_PROJECT\\_NAME.git](https://github.com/YOUR_USERNAME/YOUR_PROJECT_NAME.git).

# Initial Commit with AI Agent

**User:** "Initialize a git repo for this folder, stage all my files, and push them to a new GitHub repo at [URL]. Handle the initial commit too."

**Cursor:** "I will run the following for you:

1. git init
2. git add .
3. git commit -m 'initial commit'
4. git remote add origin [URL]
5. git push -u origin main

Click 'Run' to execute."



Your repo  
URL

# Pushing Changes to GitHub

- When you change your code, you need to update the repo as well
- You can do this without using the terminal in Cursor 😊
  - GitHub tab (click stuff)
  - AI agent (tell it stuff)
- The basic steps are
  1. Add files to be committed
  2. Commit the files with a comment describing the commit
  3. Push the commit to the repo

# Pushing Changes to GitHub



## TERMINAL WAY

```
# 1. Stage all changes
git add .

# 2. Snapshot with a message
git commit -m "added a pink vibey UI"

# 3. Ship it to the cloud
git push
```



## CURSOR UI WAY

1. Open **Source Control Tab** (Ctrl+Shift+G).
2. Click the + icon next to files to "Stage".
3. Type your update message in the box.
4. Click **Commit**.
5. Click **Sync Changes** or **Push**.



**Pro-Tip:** Use the Magic Wand in Cursor to let AI write your commit message!

# Pushing Changes with AI Agent

**User:** "I just finished the MongoDB integration and added the pink UI. Can you stage all my changes, write a commit message, and push it to GitHub?"

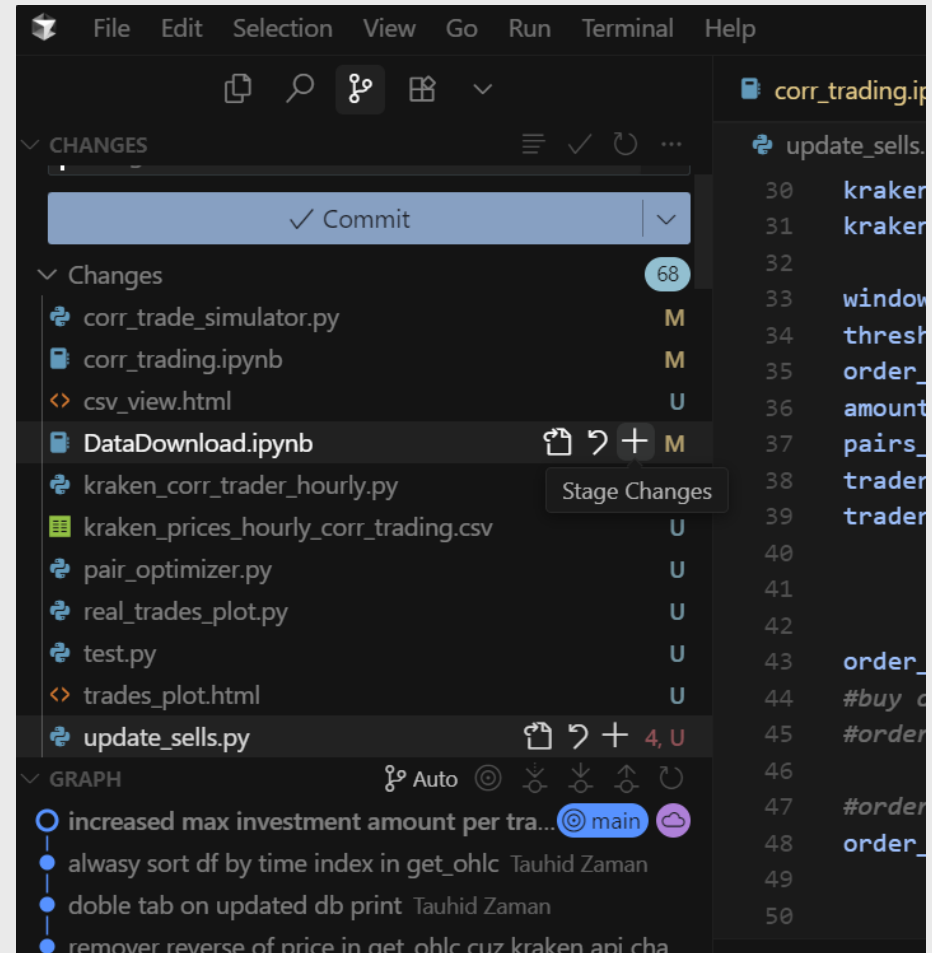
**Cursor:** "I've analyzed your changes. I will:

1. Run `git add .`
2. Commit with: `'feat: integrate MongoDB and update UI to pink theme'`
3. Run `git push origin main`

**Click 'Run' to execute these commands."**

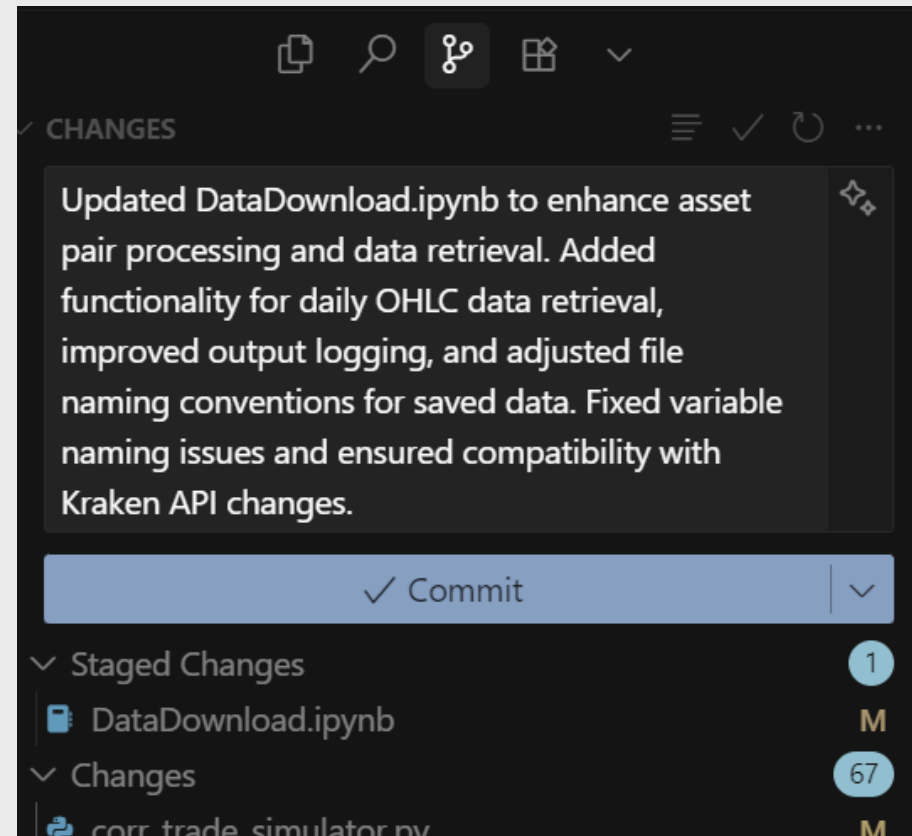
# 1. Stage Files to Be Committed

- Go into the GitHub tab
- Choose the files you want to stage for commit by clicking “+”
  - M = modified
  - U = untracked (not added to repo yet)



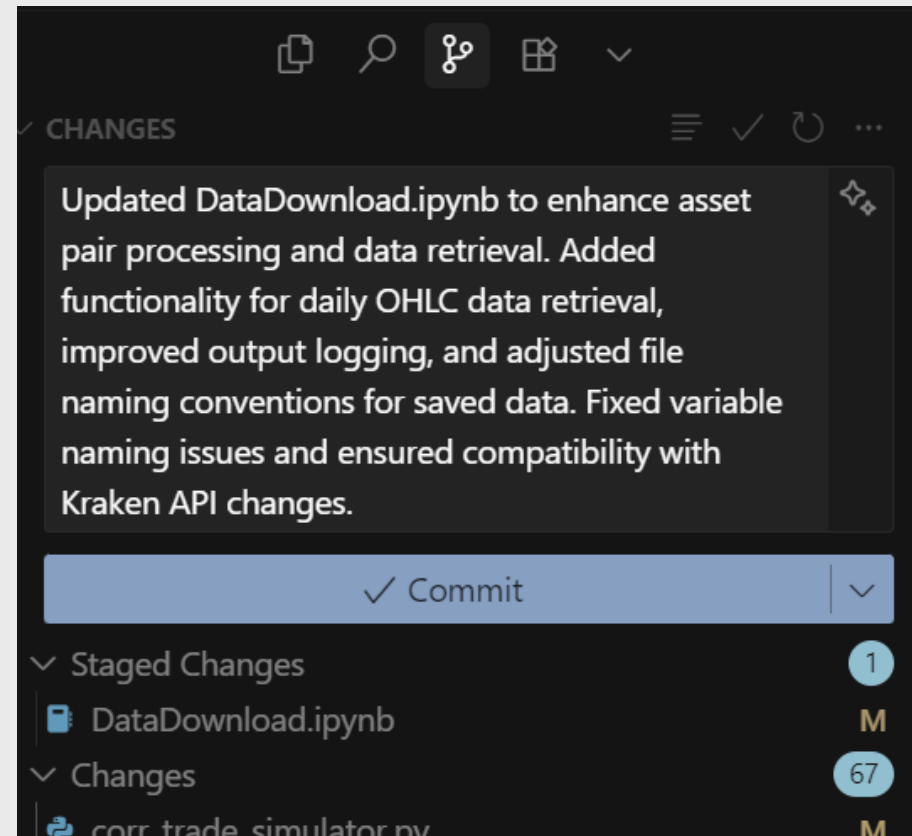
## 2. Write Commit Message

- After you add the files, they will appear in the “Staged Changes” list
- Click the sparkles button to have the AI write a commit message (you need a commit message)
- Click “Commit”



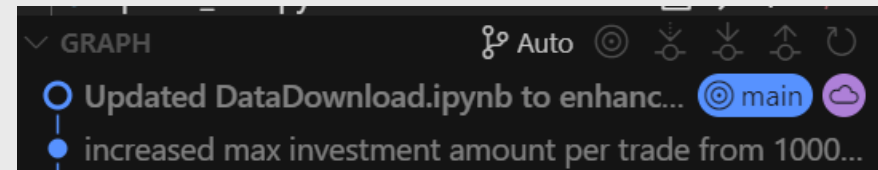
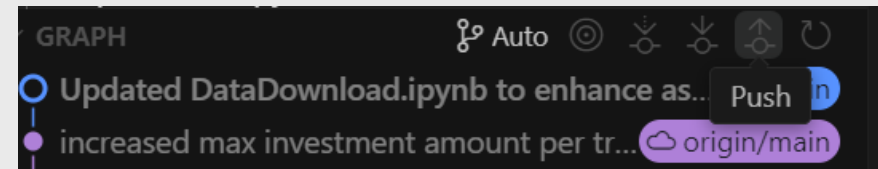
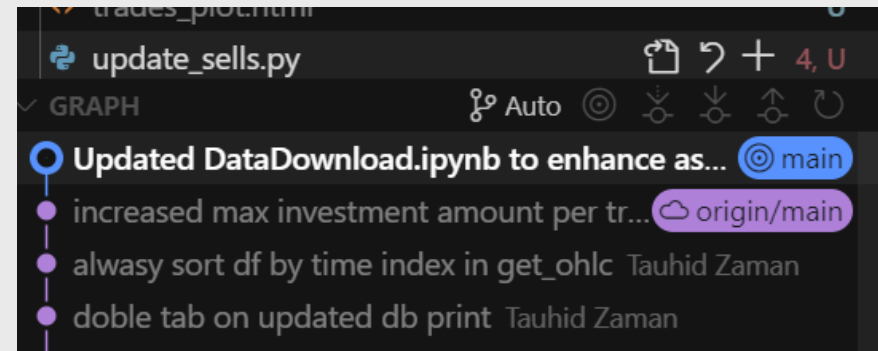
# 3. Commit Changes

- Click “Commit”



## 4. Push Commit to Repo

- In the “GRAPH” tab you will see your commit in blue
- The current repo state is in purple
- Click Push (the up arrow) to push your commit to the repo
- You will see your commit and the repo aligned after you push



# Using a Cloned Repo

- You can clone a repo and then push it to your own repo so you can deploy it
    - You will do this for your homework
1. Clone the repo (git clone <url>)
  2. Second, you delete the original remote
  3. Third, you put your repo as the new remote

# Using a Cloned Repo



## TERMINAL WAY

```
git clone [PROFESSOR_URL]
# Switch the "Home" address:
git remote remove origin
git remote add origin [YOUR_NEW_URL]
git push -u origin main
```



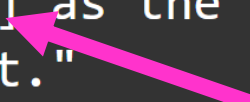
## CURSOR UI WAY

1. Click ... (Three Dots) in Git Tab
2. Go to **Remote** → **Remove Remote**
3. Go to **Remote** → **Add Remote**
4. Paste **YOUR** new Repo URL
5. Stage, Commit, and hit **Publish**

# Using a Cloned Repo with AI Agent

**User:** "I just cloned this. Please remove the current origin remote and add my new repo [YOUR\_URL] as the new origin. Then push my local main branch to it."

**Your repo  
URL**



**Cursor:** "Understood. I'll swap the remotes now."

Running: git remote remove origin

Running: git remote add origin [YOUR\_URL]

Pushing to your new home..."

# Coding Session

- **We will build a simple React app**
- **We will try to make the style look cool**
- **Give the app some simple AI functionality**